ADITYA

ENLIGHTENS THE NESCIENCE

ADITYA COLLEGE OF ENGINEERING & TECHNOLOGY

ADB ROAD, SURAMPALEM, E.G. DIST.

DEPARTMENT OF

Name: PIN No. | | | | | | | | | | |

Certified that this is the bonafide record of practical work done by

Mr. / Ms. .....

a students of ..... with PIN No.....

in the ..... Laboratory during the year .....

No of Practicals Conducted [ ] No. of Practicals Attended [ ]

Signature - Faculty Incharge Signature - Head of the Department

Submitted for the Practical examination held on.....

EXAMINER - 1 EXAMINER - 2

CONTENT

Sl.No.	Date	Experiment Title	Page No
1.		Take any real time problem and do the following experiments.	
a)		Write down the problem statement for a suggested system of relevance. Develop Flow-Charts to understand basic problem-solving technique.	
b)		Do requirement analysis and develop Software Requirement Specification Sheet (SRS) for suggested system.	
c)		Using COCOMO model estimate effort.	
d)		Perform Estimation of effort using FP Estimation for chosen system.	
e)		Analyze the Risk related to the project and prepare RMMM plan.	
f)		Develop Time-line chart and project table using PERT or CPM project scheduling methods.	
g)		Draw E-R diagrams, DFD, CFD and structured charts for the project.	
h)		Design of Test cases based on requirements and design.	
i)		Prepare FTR.	
j)		Prepare Version control and change control for software configuration items.	
2.		Using UML/JAVA, design Abstract Factory design pattern.	
3.		Using UML/JAVA, design Builder design pattern.	
4.		Using UML/JAVA, design Façade design pattern.	
5.		Using UML/JAVA, design Bridge design pattern.	
6.		Using UML/JAVA, design Decorator design pattern.	
7.		User gives a print command from a word document. Design to represent this chain of responsibility design pattern.	

Experiment – 1 : Take any real time problem and do the following experiments.

With the increasing prevalence of surveillance technology, there is a growing need to protect privacy and ensure security in private and public spaces. This project aims to develop a software solution for detecting hidden spy cameras in various environments. Leveraging principles of software engineering and design patterns, the project will deliver a robust, scalable, and efficient system for spy camera detection.

Date: _ _ _ _ _ _ _ _ _ _ _

Experiment – 1.a: Write down the problem statement for a suggested system of relevance. Develop Flow-Charts to understand basic problem solving technique.

Problem Statement

The prevalence of covert surveillance devices, such as spy cameras, poses a significant threat to privacy. This project aims to develop a software solution for detecting and alerting users of potential spy cameras in their vicinity. The solution will leverage image processing and machine learning techniques to identify hidden cameras, providing an additional layer of security for individuals in sensitive environments.

SYSTEM REQUIREMENTS

Camera Detection:

- Detect hidden cameras using infrared (IR) & radio frequency (RF) scanning

User Interface:

- Provide a user-friendly interface for initiating scans and displaying results.
- Allow users to view detected cameras' locations on a map or floor plan.

Data Management:

- Store scan history and detected camera data.
- Provide options for exporting scan reports (PDF, CSV).

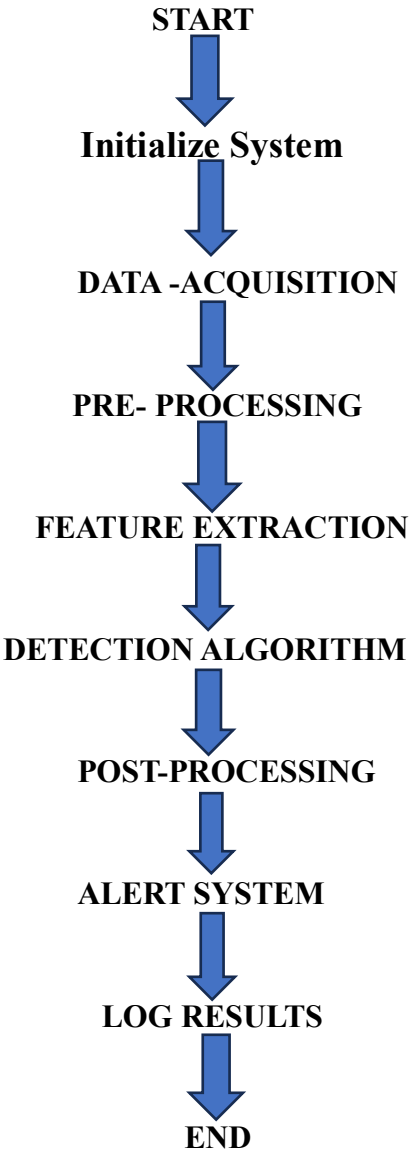
System Settings:

- Allow configuration of scan sensitivity and detection methods.
- Manage user accounts and authentication for access control

Connectivity:

- Support for Wi-Fi or Bluetooth connectivity to external scanning devices.

FLOW-CHART



This flow chart follows intermediate task management model adds additional steps for more structured task management including categorization, time estimation, delegation and periodic review.

Experiment – 1.b: Do requirement analysis and develop Software Requirement Specification Sheet (SRS) for suggested system.

Introduction:

PURPOSE

The purpose of this document is to provide a comprehensive Software Requirements Specification (SRS) for the Spy Camera Detection Project. This document outlines the functional and non-functional requirements of the system, aiming to serve as a guide for the development, testing, and validation of the project.

SCOPE

The Spy Camera Detection Project aims to develop a software application capable of detecting hidden spy cameras within a given environment. This system will utilize various detection techniques, including infrared light detection, network scanning, and image recognition to identify potential spy cameras. The application will be designed to run on multiple platforms, including Windows, macOS, and Linux.

Requirements:

Functional Requirements:

When developing a spy camera detection project, it's important to outline both functional and non-functional requirements to ensure the system meets all necessary specifications and performance criteria.

Functional Requirements

1. **Camera Detection**
   - Detect and identify various types of spy cameras (wired, wireless, mini, pinhole, etc.).
   - Provide real-time alerts when a spy camera is detected.
2. **Range of Detection**
   - Specify the effective range for detecting cameras (e.g., up to 30 feet).
3. **Multiple Detection Modes**
   - RF signal detection for wireless cameras.
   - Lens reflection detection using infrared or laser technology.
   - Visual inspection using high-resolution cameras and AI algorithms.
4. **User Interface**
   - User-friendly mobile and/or desktop application for system control and monitoring.
   - Display of detected camera locations on a map or floor plan.
   - Manual scan option for targeted detection in specific areas.

5. **Data Logging**
    - Log detection events with timestamps and location data.
    - Provide access to historical data for analysis and reporting.
6. **Alerts and Notifications**
    - Immediate alerts via push notifications, SMS, or email when a camera is detected.
    - Audible and/or visual alerts on the detection device.
7. **Power Management**
    - Battery status monitoring and low battery alerts.
    - Options for power-saving modes.
8. **Self-Test and Calibration**
    - Automated self-test routines to ensure the system is functioning correctly.
    - Manual calibration options for improved accuracy.
9. **Integration with Other Systems**
    - Compatibility with existing security systems and IoT devices.
    - API for third-party software integration.

.

Non-Functional Requirements:

Performance

- Detection accuracy: Must achieve at least 95% accuracy in detecting spy cameras.
- Latency: Detection and alert latency should be under 5 seconds.

Reliability

- System uptime: Should ensure 99.9% uptime.
- Mean Time Between Failures (MTBF): Should be greater than 1 year.

Scalability

- Ability to scale up to detect cameras in large buildings or multiple locations.
- Support for multiple detection devices connected to a single monitoring system.

Security

- Data encryption: All stored and transmitted data must be encrypted.
- User authentication: Multi-factor authentication for accessing the system.

Usability

- Intuitive user interface with minimal learning curve.
- Multilingual support for global use.

Portability

- Detection devices should be lightweight and easy to transport.
- Mobile app compatibility with both iOS and Android platforms.

Compliance

- Compliance with privacy laws and regulations in target markets.
- Adherence to relevant industry standards (e.g., FCC for RF devices).

System Features:

Multi-Mode Detection:

- RF Signal Detection: Identifies wireless spy cameras by detecting radio frequency signals.
- Lens Reflection Detection: Uses infrared or laser technology to detect the reflection from camera lenses.
- Thermal Detection: Detects heat signatures from active cameras.
- Magnetic Field Detection: Identifies magnetic fields generated by electronic devices

Real-Time Detection

- Instant Alerts: Provides immediate alerts when a spy camera is detected.
- Continuous Monitoring: Offers 24/7 monitoring capabilities for constant surveillance.

Range and Sensitivity Settings

- Adjustable detection range (e.g., short, medium, long).
- Sensitivity controls to fine-tune detection based on the environment.

External Interface

Dashboard

- Centralized dashboard for monitoring and controlling all detection activities.
- Visual representation of detected devices, including their locations on a map or floor plan.

Mobile Application

- Compatible with iOS and Android devices.
- Allows remote control and monitoring of detection devices.
- Push notifications for alerts and updates.

Desktop Application/Web Interface

Video Stream:

- The project accesses the device's camera feed using web technologies.
- It captures live video from the camera.

2. **Object Detection:**
    - The project employs the COCO-SSD (Common Objects in Context - Single Shot MultiBox Detector) model.
    - Specifically, it focuses on identifying objects resembling cell phones or mobile phones within the video stream.
3. **Real-time Monitoring:**
    - The system continuously monitors the video feed.
    - When it detects a potential spy camera (object resembling a phone), it alerts the user.
4. **User Interface:**
    - Users interact with the system through a web interface.
    - The interface displays the live camera feed and detection results.

Technical Details

Hardware and Software Requirements

- **Hardware:**
    - A computer or mobile device with a camera**.**
- **Software:**
    - A modern web browser that supports the navigator.mediaDevices.getUserMedia API.
    - TensorFlow.js library (version 3.9.0).
    - COCO-SSD model from TensorFlow.js (version 3.9.0).

System Architecture

1. Web Interface:
    - Users interact with the system through a web interface.
    - HTML, CSS, and JavaScript create and manage this interface.
2. Video Stream:
    - The system captures the live camera feed using the navigator.mediaDevices.getUserMedia API.
    - It displays the video stream in an HTML video element.
3. Object Detection:
    - It identifies objects resembling cell phones or mobile phones.
4. Monitoring:
    - Real-time monitoring occurs through JavaScript functions that repeatedly call the object detection and rendering functions.

Experiment – 1.c: Using COCOMO model estimate effort.

Basic COCOMO Model:

Assumptions

Project Size :12000 lines of code (LOC)

COCOMO Model: Organic

Error Adjustment Factor (EAF) :1.0(Neutral Adjustment)

Effort = a x (size) pow(b) x EAF

a=2.4

b=1.05

size = 12000 (Loc)

EAF = 1.0

Effort = 2.4 x (12000) pow (1.05) x 1.0

Effort = 2.4 x (10) pow (1.05)

Effort = 2.4 x 13.587

Effort = 32.610 persons-months

Time (T) = C (E) pow (d)

= 2.5 (**32.610**) pow (0.38)

= 2.5 x 7.6477

T = 9.397

INTERMEDIATE MODEL:

Project Size: 20000 lines of code(loc)

COCOMO model: Organic

Effort Adjustment Factor (EAF): Detailed Factors Considered

Effort = a x (size) pow (b) x EAF

a = 2.4

b = 1.05

EAF = 1.2

Size = 20000

Where EAF is effort is calculated as;

EAF = (Summation I = 1 to N Rating I)

Where 'n' is the total numbers of cost divers and the rating 'I' is the rating assigned to each cost divers and size is the estimated project size in KLOC.

From, Product Attribute (Nominal)

The Complexity of the product = 1.00

Hardware Attribute (High)

The Required Turn-Around Time = 1.07

Project Attributes (Low)

Use of Software Tools = 1.10

EAF = 1.00 x 1.07 x 1.10

EAF = 1.177

EFFORT (E) = 1.177 x 26.5128

E = 31.2055 Persons-Months

Time (T) = 2.5 x (31.2055) pow (0.38)

= 2.5 x 3.6966

= 9.2415

T = 9 Months

People Required (R) = E/T

R = 31.2055/9.2415

R = 3.3744

R = 3 persons

Experiment – 1.d: Perform Estimation of effort using FP Estimation for chosen system.

Function Point (FP) estimation is a method used to estimate the size, effort, and time required for software development projects. Here are the steps to perform FP estimation for your task management system:

Step 1: Identify the Functional User Requirements

1. **External Inputs (EI)**: :These are inputs that the system receives from users or external devices to perform its functions.
   - User Commands
   - Sensor Inputs
   - Environmental Data
2. **External Outputs (EO)**: These are outputs from the system. For the task management system, these could be:
   - Alerts and Notifications
   - Detection Reports
   - Status Updates
3. **External Inquiries (EQ)**: These are requests for information that the system responds to, typically from users or other systems
   - Detection History
   - System Status
   - Configuration Settings
4. **Internal Logical Files (ILF)**: These are data files that the system maintains internally to support its operations be:
   - Detection Logs
   - System Configuration Data
   - Calibration Data
   - User Profiles
5. **External Interface Files (EIF)**: These are files that the system exchanges with other systems or external devices:
   - Integration Data Files,
   - API Data Exchange
   - Backup and Restore Files

Step 2: Assign Complexity Weights

Assign weights based on the complexity (low, average, high) of each type of function.

Function Type	Weight (Low)	Weight (Average)	Weight (High)
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EQ)	3	4	6
Internal Logical Files (ILF)	7	10	15
External Interface Files (EIF)	5	7	10

Step 3: Calculate Unadjusted Function Points (UFP)

Identify the count of each function type and assign complexity.

- **External Inputs (EI):**
  - User Registration (Average) = 1 * 4 = 4
  - Task Creation (Average) = 1 * 4 = 4
  - Task Update (Average) = 1 * 4 = 4
  - Task Deletion (Average) = 1 * 4 = 4
  - Project Creation (Average) = 1 * 4 = 4
  - Category Creation (Average) = 1 * 4 = 4
- **External Outputs (EO)**:
  - Task List Display (Average) = 1 * 5 = 5
  - Project List Display (Average) = 1 * 5 = 5
  - Notification Display (Average) = 1 * 5 = 5
- **External Inquiries (EQ):**
  - Task Search (Average) = 1 * 4 = 4
  - Project Search (Average) = 1 * 4 = 4
- **Internal Logical Files (ILF):**
  - User Data (Average) = 1 * 10 = 10
  - Task Data (Average) = 1 * 10 = 10
  - Project Data (Average) = 1 * 10 = 10
  - Category Data (Average) = 1 * 10 = 10
  - Notification Data (Average) = 1 * 10 = 10
- **External Interface Files (EIF):**
  - External Calendar Integration (Low) = 1 * 5 = 5

Step 4: Compute the Unadjusted Function Points (UFP)

UFP=∑(count× weight)\text{UFP} = \sum (\text{count} \times \text{weight})UFP=∑(count× weight)
UFP=4+4+4+4+4+4+5+5+5+4+4+10+10+10+10+10+5=102\text{UFP} = 4 + 4 + 4 + 4 + 4 + 4 + 5 + 5 + 5 + 4 + 4 + 10 + 10 + 10 + 10 + 10 + 5 =
102UFP=4+4+4+4+4+4+5+5+5+4+4+10+10+10+10+10+5=102

Step 5: Calculate the Value Adjustment Factor (VAF)

Rate the 14 General System Characteristics (GSCs) on a scale of 0 to 5 and compute the VAF.

For simplicity, let's assume an average complexity rating of 3 for all GSCs.

Total Degree of Influence=14×3=42\text {Total Degree of Influence}
= 14 \times 3
= 42 Total Degree of Influence
=14×3=42
VAF=0.65+(0.01×Total Degree of Influence) =0.65+(0.01×42)
=0.65+0.42=1.07\text {VAF} = 0.65 + (0.01 \times \text {Total Degree of Influence})
= 0.65 + (0.01 \times 42) = 0.65 + 0.42
=1.07VAF=0.65+(0.01×Total Degree of Influence)
=0.65+(0.01×42)
=0.65+0.42
=1.07

Step 6: Calculate the Adjusted Function Points (AFP)

AFP=UFP×VAF
=102×1.07=109.14\text {AFP}
= \text {UFP} \times \text {VAF}
= 102 \times 1.07
= 109.14AFP
=UFP×VAF
=102×1.07
=109.14

Step 7: Estimate Effort

To estimate the effort, you need to determine the productivity rate, which is typically measured in function points per person-month. Let's assume an average productivity rate of 20 function points per person-month.

Effort=AFP Productivity Rate

=109.1420≈5.46 person-months\text {Effort}

= \frac {\text {AFP}} {\text {Productivity Rate}}

= \frac {109.14}{20} \ approximately 5.46 \text {person-months}

Effort = Productivity Rate AFP

=20109.14

≈5.46 person-months

Experiment – 1.e: Analyze the Risk related to the project and prepare RMMM plan.

Risk	Probability	Impact	Severity	Mitigation	Monitoring	Management	Estimate Cost
Privacy Concerns	High	Medium	Medium	Ensure data encryption and anonymization	Monitor data handling practices	Regular audits and adherence to privacy regulations	$3,000
Technical Failures	Low	High	High	Use reliable hardware and perform regular maintenance	Scheduled hardware inspections and performance monitoring	Maintain a supply of spare parts and tools for quick fixes	$10,000
Budget Overrun	Medium	High	High	Accurate budget planning and cost control measures	Monthly budget reviews	Adjust project scope or timeline as necessary	$2,000
Project Delays	Medium	Medium	Medium	Detailed project timeline and contingency planning	Track project progress with Gantt charts	Reallocate resources and adjust schedules as needed	$4,000
Security Vulnerabilities	Medium	High	High	Implement robust cybersecurity measures	Conduct regular security assessments	Update security protocols and patch vulnerabilities	$6,000
Inadequate Training for Personnel	Low	Medium	Medium	Comprehensive training programs	Regular assessment of staff performance	Ongoing training and development programs	$2,000
Regulatory Non-compliance	Low	High	High	Ensure adherence to relevant regulations	Regular compliance audits	Consult with legal experts and stay updated on regulations	$3,500
Vendor Reliability	Low	Medium	Medium	Choose reputable vendors and have backup options	Evaluate vendor performance regularly	Maintain good relationships and have contingency plans	$1,500
False Positives	Medium	High	High	Implement advanced algorithms to reduce false positives	Regularly review detection logs and adjust algorithms	Conduct routine testing and improvements	$5,000

Date: _ _ _ _ _ _ _ _ _ _

Risk Monitoring and Review:

Regularly review identified risks, report on their status, and develop contingency plans for high-impact risks. Implement a system for continuous risk assessment and communication with stakeholders.

Estimated Cost:

The estimated cost associated with addressing these risks depends on various factors such as the resources required for mitigation strategies, potential impact on project timelines, and the complexity of risk management activities. It's essential to allocate a contingency budget to cover unforeseen expenses related to risk mitigation.

Overall, incorporating risk analysis, RMMM planning, and estimated costs into the project budget ensures that potential risks are proactively managed, minimizing their impact on project success and financial outcomes.

Experiment – 1.f: Develop Time-line chart and project table using PERT or CPM project scheduling methods.

To estimate the total cost for RMMM, we will sum up the costs for each mitigation strategy

Let's assumed cost for RMMM is $90,000

RMMM Cost Estimation = Cost Estimation of each risk/Assumed Cost Of RMMM

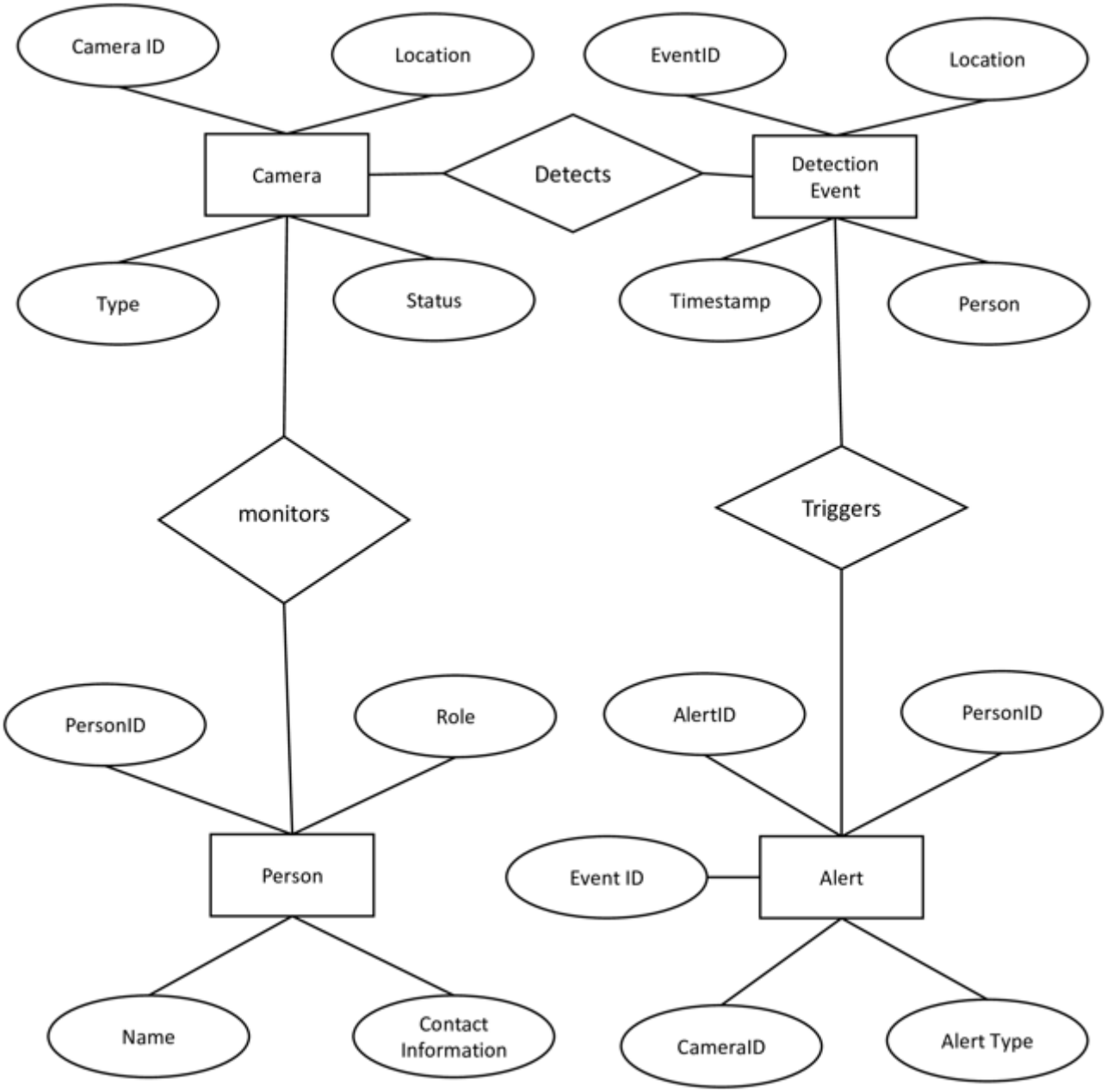
$$\text{RMMM} = \frac{\$50000 + \$10000 + \$8000 + \$7000 + \$12000 + \$6000 + \$9000 + \$4000 + \$8000 + \$3000 + \$10000 + \$7000}{\$90,000}$$

RMMM = $ 89,000 / $ 90,000

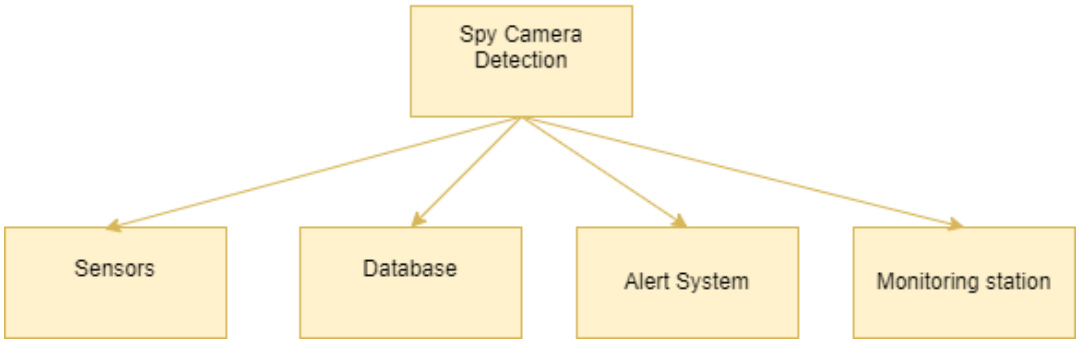
RMMM = 0.9888

Experiment – 1.g: Draw E-R diagrams, DFD, CFD and structured charts for the project.

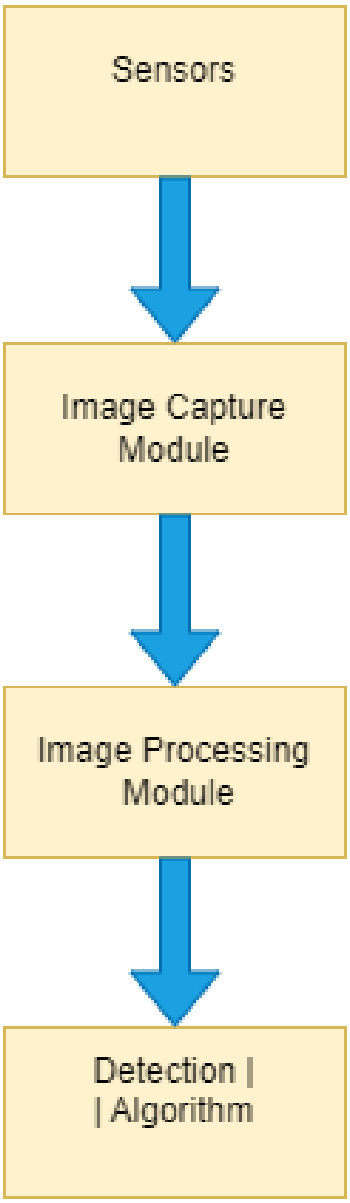
ER DIAGRAM



DFD-Level 0


Spy Camera Detection
Sensors
Database
Alert System
Monitoring station

DFR-Level 1

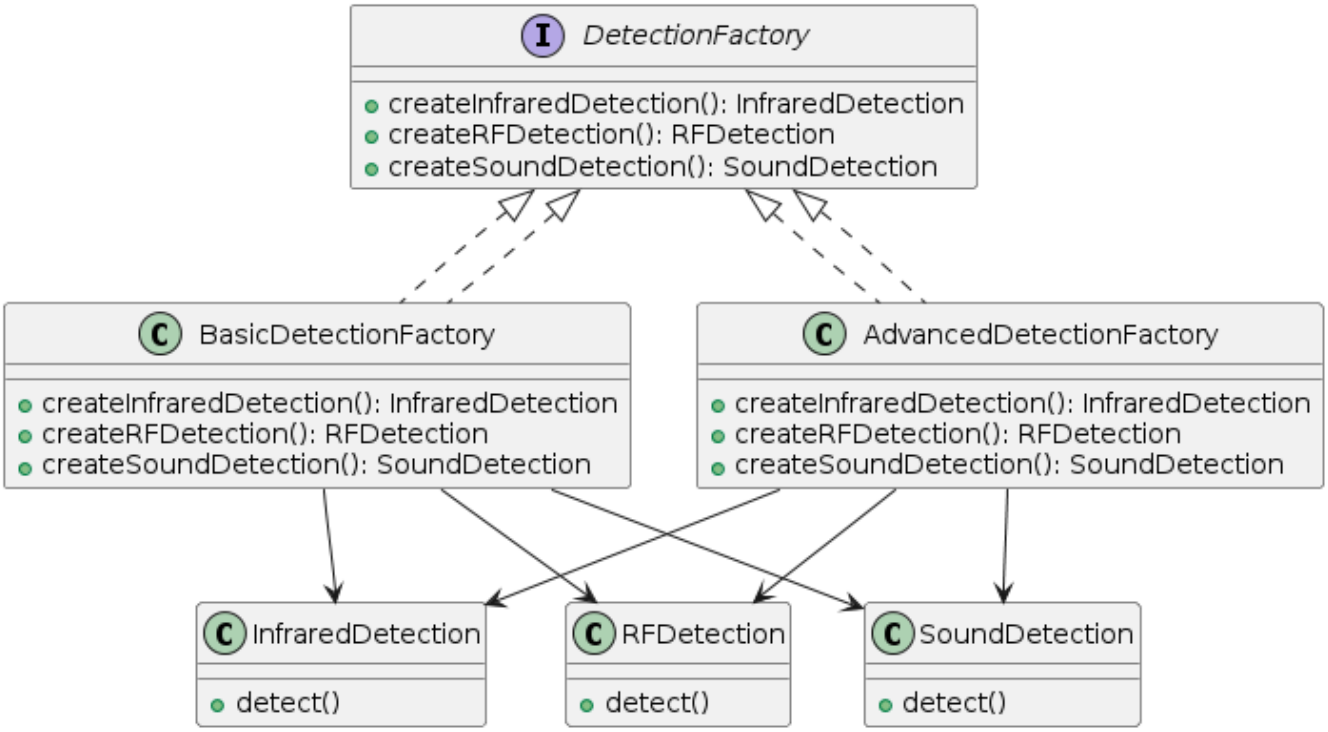

Sensors
Image Capture Module
Image Processing Module
Detection | | Algorithm

Experiment – 1.h: Design of Test cases based on requirements and design.

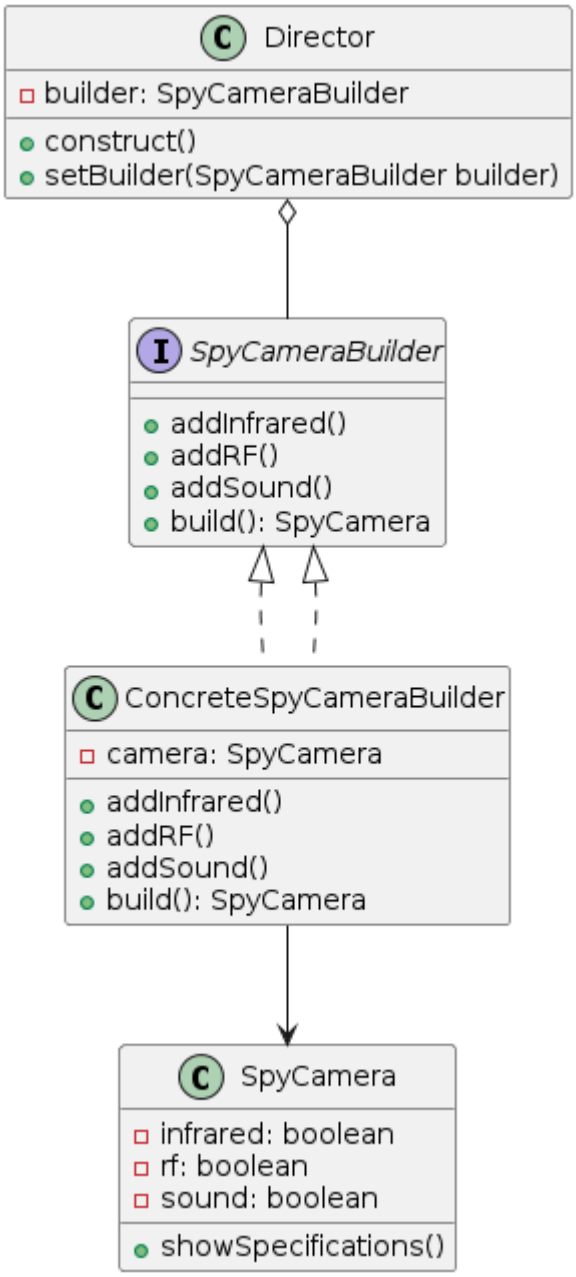
Module	Test Case ID	Description	Preconditions	Test Steps	Expected Results	Actual Results	Status
User Management	UM001	User login with valid credentials	User account exists	1. Open the application. 2. Enter valid credentials. 3. Click login.	User successfully logs in.	User successfully logged in.	Pass
User Management	UM002	User login with invalid credentials	User account exists	1. Open the application. 2. Enter invalid credentials. 3. Click login.	Login fails with an error message.	Login failed with an error message.	Pass
User Management	UM003	User logout	User is logged in	1. Click the logout button.	User is logged out and redirected to the login screen.	User was logged out and redirected to login screen.	Pass
Device Management	DM001	Add new detection device	None	1. Navigate to the device management section. 2. Add a new device. 3. Enter device details and save.	New device is added successfully.	New device was added successfully.	Pass
Device Management	DM002	Remove detection device	Device exists in the system	1. Navigate to the device management section. 2. Select a device. 3. Remove the device.	Device is removed successfully.	Device was removed successfully.	Pass
Detection Functionality	DF001	Detect wired spy camera	Camera is connected to power supply	1. Switch on the detector. 2. Sweep the detector around the room.	The detector identifies the presence of a wired camera	The detector identified a wired camera.	Pass
Detection Functionality	DF002	Detect wireless spy camera	Camera is turned on and transmitting	1. Switch on the detector. 2. Sweep the detector around the room.	The detector identifies the presence of camera	The detector did not identify a wireless camera.	Fail

Date: _ _ _ _ _ _ _ _ _ _ _

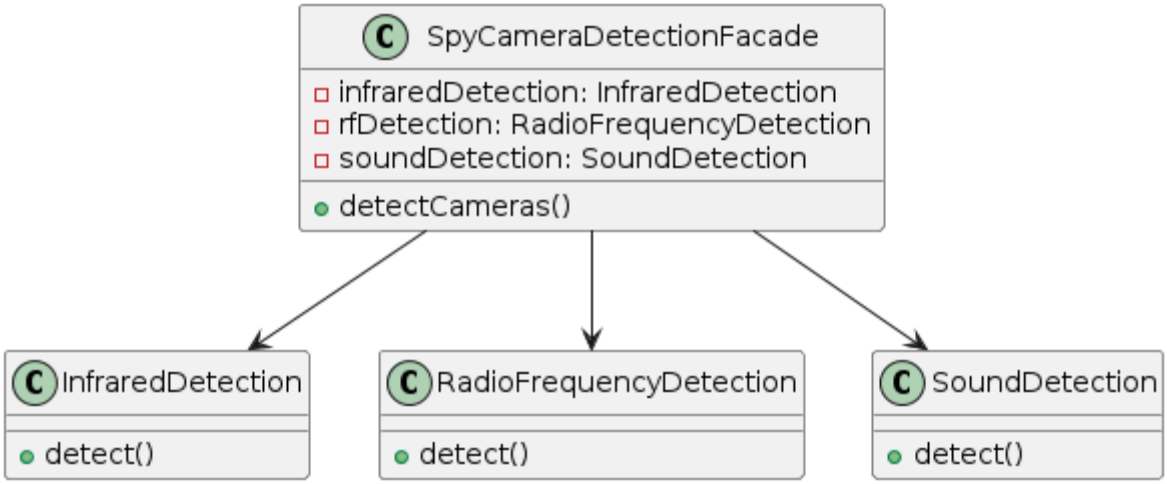
Experiment – 2: Using UML/JAVA, design Abstract Factory design pattern.



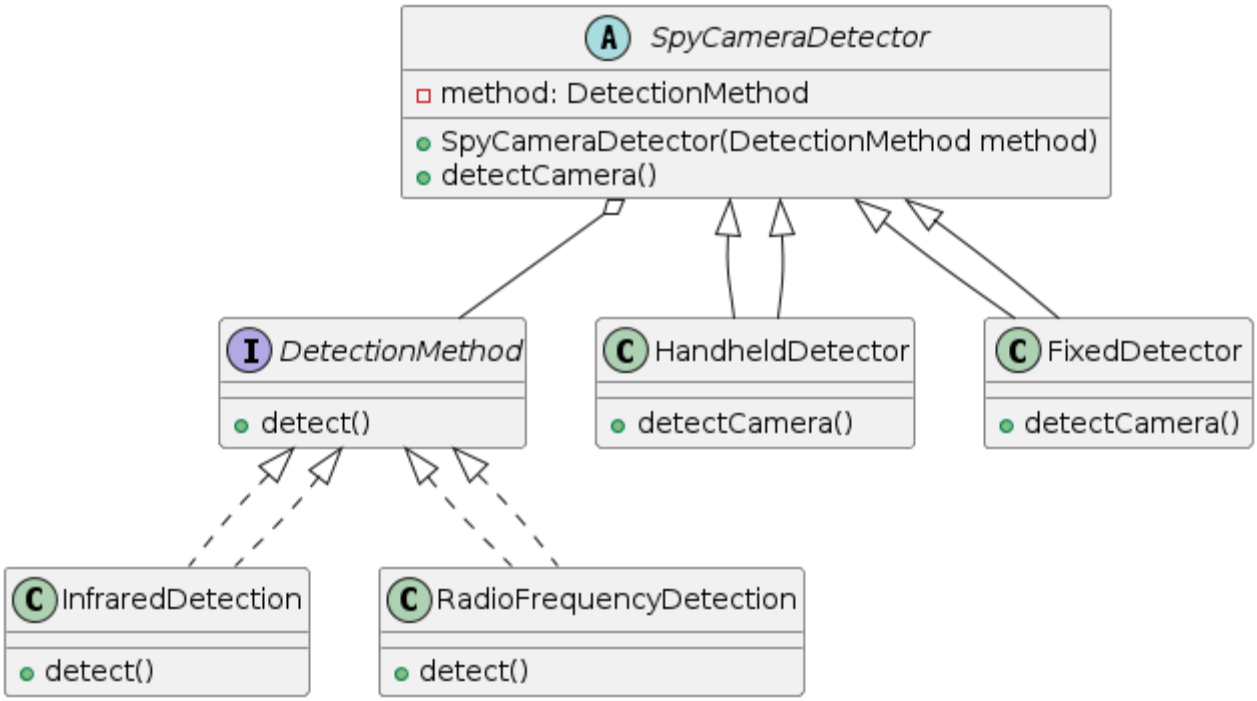
Experiment – 3: Using UML/JAVA, design Builder design pattern.



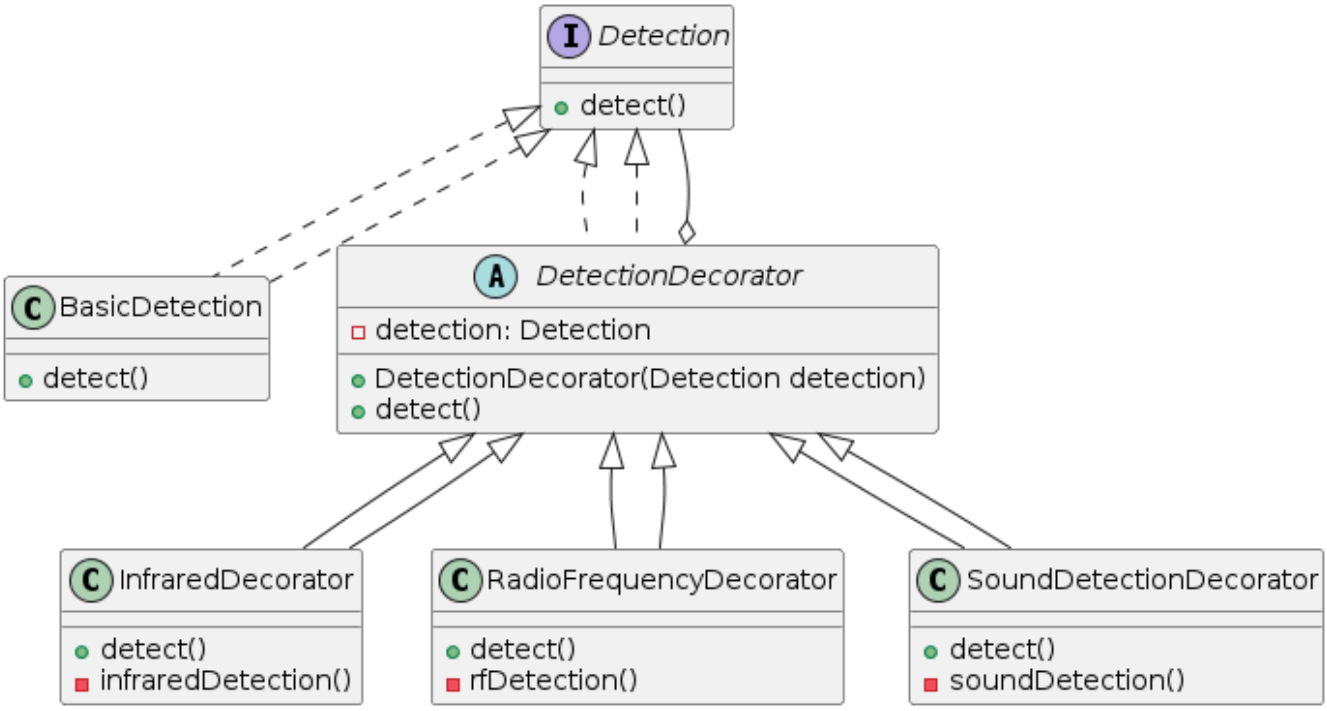
Experiment – 4: Using UML/JAVA, design facade design pattern.



Experiment – 5: Using UML/JAVA, design Bridge design pattern.



Experiment – 6: Using UML/JAVA, design Decorator design pattern.



Date: _ _ _ _ _ _ _ _ _ _ _